

Gabarito e Explicações - Dia 4

Aquecimento

1) A Primeira Função

```
print("Funções reduzem repetição e organizam o código.")
```

```
def saudacao():  
    print("Olá, bem-vindo!")
```

```
saudacao()
```

Explicação: As funções isolam lógica para uso futuro. Definimos com def. Para que ela execute, não basta apenas declará-la, devemos realizar a chamada da função explicitamente no código.

2) Parâmetros Simples

```
def exibir_dobro(n: int):  
    print(n * 2)
```

```
valor = int(input("Digite um número: "))  
exibir_dobro(valor)
```

Explicação: O parâmetro n: int obriga (tipa) a função a esperar uma informação quando for chamada. O que enviarmos nos parênteses da chamada irá preencher essa variável internamente.

3) Função com Retorno

```
def somar(a: int, b: int) -> int:  
    return a + b
```

```
resultado = somar(10, 15)  
print("A soma é:", resultado)
```

Explicação: Ao contrário do uso de print, o return permite que o valor saia da função para ser reutilizado. A setinha -> int indica explicitamente ao programador qual será o formato da saída.

4) Retorno Booleano

```
def verificar_par(n: int) -> bool:  
    if n % 2 == 0:  
        return True  
    else:
```

```
        return False

# Ou de forma mais pythonica:
# return n % 2 == 0
```

Explicação: Retornar tipos booleanos é essencial para integrar funções com condicionais if/else. A divisão com resto (módulo) determina essa característica.

5) Proteção de Escopo

```
def teste_escopo():
    mensagem = "Dentro da função"
    print(mensagem)

mensagem = "Fora da função"
teste_escopo()
print(mensagem)
```

Explicação: Variáveis criadas dentro do def morrem assim que o código da função acaba e são protegidas contra modificações. O print externo confirma que a variável principal continuou intacta.

Avançando

6) Multiplicação de Texto

```
def multiplicar_texto(texto: str, vezes: int) -> str:
    return texto * vezes

print(multiplicar_texto("Python ", 3))
```

Explicação: Em Python, o operador matemático de multiplicação *, quando aplicado entre uma string e um número, repete e concatena a mesma string de forma automática.

7) Redução de Lista

```
def somar_lista(lista: list[int]) -> int:
    soma = 0
    for num in lista:
        soma += num
    return soma
```

Explicação: Podemos passar listas inteiras como parâmetro. O laço percorre todos os itens somando ao acumulador soma e depois o retorna finalizado ao chamador.

8) Classificador de Idades

```
def classificar_idade(idade: int) -> str:
    if idade < 12:
```

```
        return "Criança"
    elif idade < 18:
        return "Adolescente"
    else:
        return "Adulto"
```

Explicação: Quando uma função executa o comando return, a sua operação é cancelada imediatamente. Por causa disso, nem precisamos de else obrigatório, mas a estrutura completa deixa mais legível.

9) Encapsulando um Laço

```
def tabuada(n: int):
    for i in range(1, 11):
        print(f"{n} x {i} = {n * i}")
```

```
tabuada(7)
```

Explicação: Ao transformar uma lógica rotineira em função, basta chamarmos tabuada() para calcular qualquer número, ocultando o processamento complexo do for.

10) Calculadora de Descontos

```
def calcular_desconto(valor: float, desconto: float) -> float:
    desconto_reais = valor * (desconto / 100)
    valor_final = valor - desconto_reais
    return valor_final
```

Explicação: Funções servem muito bem como calculadoras de regras de negócio isoladas. Entregamos os parâmetros básicos e ela nos devolve o valor já com a transformação.

Desafios

11) Primos

```
def eh_primo(num: int) -> bool:
    if num < 2:
        return False
    for i in range(2, num):
        if num % i == 0:
            return False
    return True
```

Explicação: Agora a lógica fica contida. O primeiro retorno False encerra a função caso achemos algum divisor. Se o laço for concluído sem interrupções, obrigatoriamente ele cai no return True.

12) Função Filtro de Elementos

```
def filtrar_pares(lista: list[int]) -> list[int]:
    pares = []
    for num in lista:
        if num % 2 == 0:
            pares.append(num)
    return pares
```

Explicação: A função processa uma lista e retorna outra. Instanciamos a matriz pares localmente para não poluir o arquivo principal, mantendo o processo limpo e livre de variáveis globais.

13) Calculadora Completa (Match-Case)

```
def calculadora(n1: float, n2: float, op: str) -> float:
    match op:
        case "+":
            return n1 + n2
        case "-":
            return n1 - n2
        case "*":
            return n1 * n2
        case "/":
            if n2 == 0:
                print("Erro: Divisão por Zero!")
                return 0.0
            return n1 / n2
        case _:
            print("Operação inválida.")
            return 0.0
```

Explicação: O match-case é muito mais organizado que vários elifs para verificação de strings exatas. Tratamos e isolamos a quebra por divisão de valor nulo no case "/".

14) Inversão de String

```
def inverter_string(palavra: str) -> str:
    invertida = ""
    for i in range(len(palavra) - 1, -1, -1):
        invertida += palavra[i]
    return invertida
```

Explicação: Varremos os índices da string de trás para frente, desde a última posição (len - 1) até o limite do zero (colocamos -1 na parada exclusiva do range) e concatenamos passo a passo.